C++ 基础语法

第一课 / 让程序读入、判断、处理数据

今天要写出的程序

输入：3 个分数

计算：总分与平均分

判断：是否达到 60 分

输出：结果

你已经会用 Python 怎么写?

第一份完整程序

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, C++!\n";
    return 0;
}
```

入口是 main，语句用分号结束

Python 与 C++ 的表达方式

```cpp
// Python: score = 85
int score = 85;

// Python: if score >= 60:
if (score >= 60) {
    std::cout << "pass\n";
}
```

类型写在变量前，花括号标记语句块

语句块与变量的作用域

```cpp
int score = 85;
{
    int bonus = 5;
    score += bonus;
}
std::cout << score << '\n';
// bonus is no longer in scope here
```

内层可以使用外层变量，外层不能使用已离开的局部名字

编译与运行

```
std::cout << "Hello\n"
```

少了一个分号。这次会输出 Hello 吗?

第一条诊断的位置

先看文件名和行号

回到该行，并检查前一行

修复第一个原因，再重新编译

std:: 是名字的归属

```cpp
std::cout << "AIM\n";

using std::cout;
cout << "AIM\n";
```

两种写法指向同一个标准库对象

输出的拼接顺序

```cpp
int score = 85;
std::cout << "score=" << score << '\n';
```

屏幕上会出现什么?

输入与类型

同一份数据，先确定它是什么

变量的类型

```
int count = 3;
double average = 82.5;
char grade = 'A';
std::string name("anon");
bool passed = true;
```

整数 / 小数 / 一个字符 / 字符串 / 条件

初始化与后续赋值

```cpp
int count = 3;  // initialization
count = 5;      // assignment

double measured = 3.5;
int truncated = static_cast<int>(measured);
```

初始化建立对象，赋值改变已有对象
显式转换也可能丢失小数部分

类型决定可表示的值

```cpp
int whole = 85;
double fraction = 85.5;

std::cout << sizeof(whole) << '\n';
std::cout << sizeof(fraction) << '\n';
```

整数有范围，浮点数有精度限制
类型大小依赖实现，不能凭一次运行定规则

输入按顺序进入变量

```cpp
int a = 0, b = 0;
std::cin >> a >> b;
std::cout << a - b << '\n';
```

输入 8 4，输出多少？换成两行呢?

提取运算与一整行输入

```
std::string first, second;
std::cin >> first >> second;
// Input: poppin party
// first = "poppin", second = "party"
```

按空白分词，getline 读取一整行

输入失败的分支

```cpp
int score = 0;
if (!(std::cin >> score)) {
    std::cout << "invalid input\n";
    return 1;
}
```

输入 abc，会走到哪里?

练习：读取并输出

读入姓名与分数

输出 anon:85

输入样例：anon 85

读取并输出：一起核对

```cpp
std::string name;
int score = 0;
std::cin >> name >> score;
std::cout << name << ':' << score << '\n';
```

检查变量类型、输入顺序和输出符号

运算与判断

先预测结果，再解释类型

3 / 2 的结果

```
std::cout << 3 / 2 << '\n';
std::cout << 3 / 2.0 << '\n';
std::cout << 3 % 2 << '\n';
```

三行各输出什么?

转换发生的时机

```cpp
int a = 3, b = 2;

double x = a / b;
double y = static_cast<double>(a) / b;
```

x 是 1，y 是 1.5

接收结果的类型不会重算除法

三个除法表达式的计算过程

```
3 / 2                           // 1
static_cast<double>(3) / 2      // 1.5
static_cast<double>(3 / 2)      // 1.0
```

先确定操作数类型，再执行除法，最后转换结果

赋值与比较

```cpp
int x = 10;
x = 3;

std::cout << (x == 3) << '\n';
std::cout << (x != 3) << '\n';
```

= 改值，== 问是否相等

复合条件与短路

```cpp
if (total != 0 && score / total > 0.6) {
    std::cout << "pass\n";
}
```

total 为 0 时，右边会计算吗?

与、或、非的含义

```
true  && false   // false
true  || false   // true
!true            // false
!(score < 0 || score > 100)
```

最后一行表示分数在 [0, 100] 内

变量的更新

```
int count = 3;
++count;
count += 2;
count *= 3;
```

每行之后 count 分别是多少?

数学函数与返回类型

```cpp
#include <cmath>

std::sqrt(9.0);   // 3.0
std::abs(-3);     // 3
std::floor(2.8);  // 2.0
std::ceil(2.1);   // 3.0
```

调用产生一个值，想显示它仍要输出

if 的互斥分支

```cpp
if (score >= 90) {
    std::cout << "A\n";
} else if (score >= 60) {
    std::cout << "pass\n";
} else {
    std::cout << "retry\n";
}
```

score 是 95，会打印几行?

分支顺序与边界样例

```
// Test cases for score:
// 59  -> retry
// 60  -> pass
// 89  -> pass
// 90  -> A
// 100 -> A
```

边界两侧各测一次，比只测 80 更能发现错误

练习：平均分与及格判断

读入 3 个整数分数

输出平均分，保留 1 位小数

平均分 ⩾ 60 时再输出 pass

平均分：分离计算与显示

```
double average = (a + b + c) / 3.0;
std::cout << std::fixed
          << std::setprecision(1)
          << average << '\n';
if (average >= 60) {
    std::cout << "pass\n";
}
```

显示到 1 位小数，不改变 average 存储的数值

课间休息

10 分钟

回来后处理字符串，再整合成完整程序

循环与一组数据

把重复的代码交给循环

for 的执行顺序

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
    sum += i;
}
```

初始化一次，判断后执行，最后更新

一次循环的状态表

i	condition	sum after body
1	true	1
2	true	3
3	true	6
4	false	6

最后一次判断也会发生，但不会执行循环体

循环的边界

```cpp
for (int i = 0; i < 3; ++i) {
    std::cout << i << ' ' ;
}
```

输出 0 1 2

条件改成 <= 3，会多出什么?

continue 与 break

```cpp
for (int i = 0; i < 5; ++i) {
    if (i == 1) continue;
    if (i == 4) break;
    std::cout << i << ' ' ;
}
```

输出 0 2 3

continue 跳过本轮，break 结束循环

数组的下标

```cpp
int scores[5] = {94, 87, 50};

std::cout << scores[0] << '\n';
std::cout << scores[4] << '\n';
```

5 个位置：0、1、2、3、4
剩余元素初始化为 0

初始化方式影响元素的值

```
int a[3] = {0};         // 0, 0, 0
int b[3] = {5};       // 5, 0, 0
int c[3];           // uninitialized local ints
```

未初始化的局部元素不能直接当作有效值读取

std::array 的整体赋值

```
std::array<int, 3> a = {94, 87, 50};
std::array<int, 3> b = {};
b = a;
b[0] = 100;
```

a[0] 仍是 94

原生数组不能直接写 b = a

拷贝后是两组独立元素

```
a:  94  87  50
b:  94  87  50   // after b = a

b[0] = 100

a:  94  87  50
b: 100  87  50
```

值复制建立新的对象；后续修改不会自动同步

范围循环中的副本

```
std::array<int, 3> scores = {60, 70, 80};

for (int score : scores) {
    score += 5;
}
```

循环后 scores[0] 是 60 还是 65?

用引用修改元素

```
for (int& score : scores) {
    score += 5;
}
```

scores 变成 65、75、85

& 在这里表示元素的别名

三种遍历写法

```
for (int value : scores)       // copy
for (int& value : scores)      // modify element
for (const int& value : scores) // read only
```

先决定是否修改原元素，再选择循环变量类型

练习：统计及格人数

给定 {94, 87, 50, 60, 0}

统计 ⩾ 60 的元素个数

再求平均分，预期为 58.2

统计问题的两个累加器

```
int sum = 0, passed = 0;
for (int score : scores) {
    sum += score;
    if (score >= 60) ++passed;
}
double average =
    static_cast<double>(sum) / scores.size();
```

这组固定数据：sum=291，passed=3，average=58.2

循环与数组检查

为什么 double x = 3 / 2 得不到 1.5?

为什么 score += 5 可能没有改动数组?

字符串

在一串名字里定位和修改内容

字符串也有下标

```
std::string name("soyo");
std::cout << name.size() << '\n';
name[0] = 'S';
```

size 是 4，修改后是 Soyo

find 返回位置

```
std::string members("soyo,anon,taki");
std::string::size_type pos = members.find("anon");
```

anon 从哪个下标开始?

没找到时是 npos

```
std::string::size_type pos = members.find("rana");
if (pos != std::string::npos) {
    std::cout << pos << '\n';
}
```

位置 0 是成功，不能用 if(pos) 判断

查找的三个边界

```
members = "soyo,anon,taki";
find("soyo")  // 0: found at the start
find("taki")  // 10: found later
find("rana")  // npos: not found
```

起点、非起点、未找到，都要覆盖

replace 改变后续位置

```
std::string::size_type pos = members.find("anon");
if (pos != std::string::npos) {
    members.replace(pos, 4, "rana");
}
```

结果是 soyo,rana,taki
换成更长名字后，要重新查找后续位置

追加与删除

```
members += ",tomori";
members.insert(0, "rana,");
members.erase(0, 5);
```

删除名字时，分隔符由谁处理?

字符串修改后的状态

```
start:    soyo,anon,taki
append:   soyo,anon,taki,tomori
insert:   rana,soyo,anon,taki,tomori
erase:    soyo,anon,taki,tomori
```

每一步都在上一份结果上继续修改

练习：查找并替换成员

起始：soyo,anon,taki

找到 anon 时替换为 tomori

找不到时保持原样

整合与下一课

数据开始属于同一个对象

一条记录

```
struct BandRecord {
    std::string guitar;
    std::string vocal;
};

BandRecord band = {"mutsumi", "hatsune"};
std::cout << band.guitar << '\n';
```

类型描述结构，对象保存一份具体数据

类型与两份记录

```
BandRecord a = {"mutsumi", "hatsune"};
BandRecord b = {"anon", "tomori"};
a.guitar = "soyo";
```

同一类型的两个对象，各自保存自己的字段

综合：成绩报告

读取 3 个分数并存入 std::array

循环求和，输出 1 位小数的平均分

输出及格人数

综合程序的检查顺序

94 87 50：平均分 77.0，及格 2 人

60 60 60：平均分 60.0，及格 3 人

0 0 0：平均分 0.0，及格 0 人

出错时依次检查输入、累计值、除法和比较

第一课回看

读到一个新程序，你会先看哪里?

算错结果，你会先检查什么?

下一课：函数怎样改变调用者的数据?